

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS	SESSION 2026
Épreuve E5 - Conception et développement d'applications (option SLAM)	
ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle (recto)	

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE		N° réalisation : 1
Nom, prénom : Geffroy Maxime		N° candidat : 02543646198
Épreuve ponctuelle ☒	Contrôle en cours de formation ☐	Date : 15 / 04 /2026
Organisation support de la réalisation professionnelle		
Intitulé de la réalisation professionnelle : Book By Click		
Période de réalisation : 2025 – 2026 Lieu : ESNA – Bruz Ker Lann		
Modalité : ☐ Seul(e) ☒ En équipe		
Compétences travaillées ☒ Concevoir et développer une solution applicative ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative ☒ Gérer les données		
Conditions de réalisation⁵ (ressources fournies, résultats attendus) Les petits salons, coachs indépendants, artisans et auto-entrepreneurs souhaitant travailler seuls souhaitent moderniser leur gestion des rendez-vous. Actuellement, ceux-ci sont souvent pris par téléphone, SMS ou messages, ce qui entraîne des erreurs de planning, des oublis et une perte de temps importante. La solution proposée permet de gérer les rendez-vous de manière numérique et centralisée, avec un accès simple aux créneaux disponibles pour les clients et une vision claire du planning pour le professionnel. Le résultat prend la forme d'une application web sur laquelle le professionnel peut définir ses disponibilités et configurer une semaine type avec des créneaux récurrents. Chaque créneau correspond à un seul rendez-vous, ce qui garantit qu'aucune double réservation n'est possible. Un client peut se créer un compte afin de rechercher des entreprises correspondant à ses besoins et consulter ses rendez-vous. Il peut ensuite sélectionner un créneau disponible et réserver un rendez-vous directement en ligne. Une fois la réservation effectuée, celle-ci est enregistrée dans l'application et un e-mail de confirmation est automatiquement envoyé au client. En cas d'annulation, un e-mail est également transmis afin d'assurer la traçabilité des échanges. Le professionnel peut consulter l'ensemble des rendez-vous, en ajouter manuellement si nécessaire, ou en annuler. L'application permet également de gérer des événements bloquants ou exceptionnels, tels que des congés, des fermetures ou des indisponibilités ponctuelles, qui viennent modifier automatiquement les créneaux proposés. Le professionnel dispose d'un espace d'administration lui permettant de gérer ses créneaux, ses réservations et ses paramètres. L'objectif est de fournir une application simple, destinée à des utilisateurs non techniques, permettant de fiabiliser la prise de rendez-vous et d'améliorer l'organisation quotidienne.		

⁵ En référence aux *conditions de réalisation et ressources nécessaires* du bloc « Conception et développement d'applications » prévues dans le référentiel de certification du BTS SIO.

Description des ressources documentaires, matérielles et logicielles utilisées⁶

Le développement de l'application s'appuie sur un ensemble de ressources matérielles et logicielles adaptées à la conception et au déploiement d'une application web.

Concernant les ressources matérielles, un poste de travail de type PC est utilisé pour le développement. Un serveur de test local permet de tester l'application durant les phases de conception. Un VPS est également utilisé afin d'héberger et stocker le projet.

Sur le plan logiciel, l'environnement de développement repose sur Visual Studio Code comme éditeur de code. L'outil pgAdmin est utilisé pour l'administration et la gestion de la base de données.

La partie front-end est développée avec React 19, un framework JavaScript permettant de créer des interfaces dynamiques. Le projet utilise Vite comme outil de build et de serveur de développement, Tailwind CSS 4 pour la mise en forme de l'interface, ainsi que React Router 7 pour la gestion de la navigation entre les pages.

La partie back-end est développée avec Flask en mode API, un framework web Python léger, associé à SQLAlchemy, un ORM facilitant les interactions avec la base de données. La base de données utilisée est PostgreSQL 15.

Les langages de programmation employés sont JavaScript pour la partie front-end et Python pour la partie back-end.

Enfin, plusieurs outils complètent l'environnement de travail. GitHub est utilisé pour le versionnage et la gestion du code source. Docker et Docker Compose permettent de conteneuriser l'application et de garantir des environnements cohérents. ESLint est utilisé pour améliorer la qualité du code et assurer le respect des bonnes pratiques.

Modalités d'accès aux productions⁷ et à leur documentation⁸

Les productions réalisées dans le cadre de ce projet sont accessibles via un dépôt GitHub public intitulé <https://github.com/TISEPSE/Book-By-Click>. Ce dépôt contient l'intégralité du code source de l'application ainsi que les fichiers nécessaires à son installation et à son exécution en environnement local.

La documentation du projet est mise à disposition en ligne et centralisée sur un site dédié (tiseipse.github.io/Documentation-BBC/). Elle présente le contexte, les objectifs et les fonctionnalités principales de l'application, ainsi que des guides d'installation et d'utilisation.

L'interface web de l'application est accessible en environnement local sur le port 5173.

L'API REST est accessible en environnement local sur le port 5000.

⁶ Les réalisations professionnelles sont élaborées dans un environnement technologique conforme à l'annexe II.E du référentiel du BTS SIO.

⁷

⁸

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

La réalisation professionnelle présentée consiste en le développement d'une application web de gestion de rendez-vous en ligne nommée Book by Click. Ce projet s'inscrit dans un contexte où de nombreux indépendants, artisans, coachs et petits salons gèrent encore leurs rendez-vous de manière informelle (agenda papier, téléphone, SMS, messagerie instantanée), ce qui entraîne des erreurs de planning, des oublis, une perte de temps et un manque de traçabilité.

L'objectif de Book by Click est de proposer une solution numérique centralisée permettant aux professionnels de définir leurs disponibilités, de configurer une semaine type avec des créneaux récurrents et de rendre ces créneaux accessibles aux clients via une interface web. Chaque créneau correspond à un seul rendez-vous, garantissant l'absence de double réservation. Les clients peuvent créer un compte, rechercher des entreprises, consulter leurs rendez-vous et réserver directement en ligne un créneau disponible.

Une fois la réservation effectuée, celle-ci est enregistrée dans la base de données et un e-mail de confirmation est automatiquement envoyé au client. En cas d'annulation, un e-mail est également transmis afin d'assurer la traçabilité des échanges. Le professionnel peut consulter l'ensemble des rendez-vous, en ajouter manuellement si nécessaire et en annuler depuis son espace d'administration. L'application permet également la gestion d'événements bloquants ou exceptionnels (congs, fermetures, indisponibilités ponctuelles) qui viennent automatiquement impacter les créneaux proposés.

La solution repose sur une architecture 3-tiers, garantissant une séparation claire des responsabilités :

- Couche présentation : développée en React 19, avec Vite comme outil de build, Tailwind CSS 4 pour le design et React Router 7 pour la navigation. Cette couche assure l'affichage des interfaces, la saisie des données par les utilisateurs et la communication avec l'API.
- Couche applicative : développée en Python avec le framework Flask en mode API REST. Elle gère l'authentification, la logique métier, la validation des données, la génération des créneaux, le traitement des réservations et l'envoi des e-mails.
- Couche de persistance : basée sur une base de données relationnelle PostgreSQL 15, assurant le stockage structuré, sécurisé et cohérent des données.

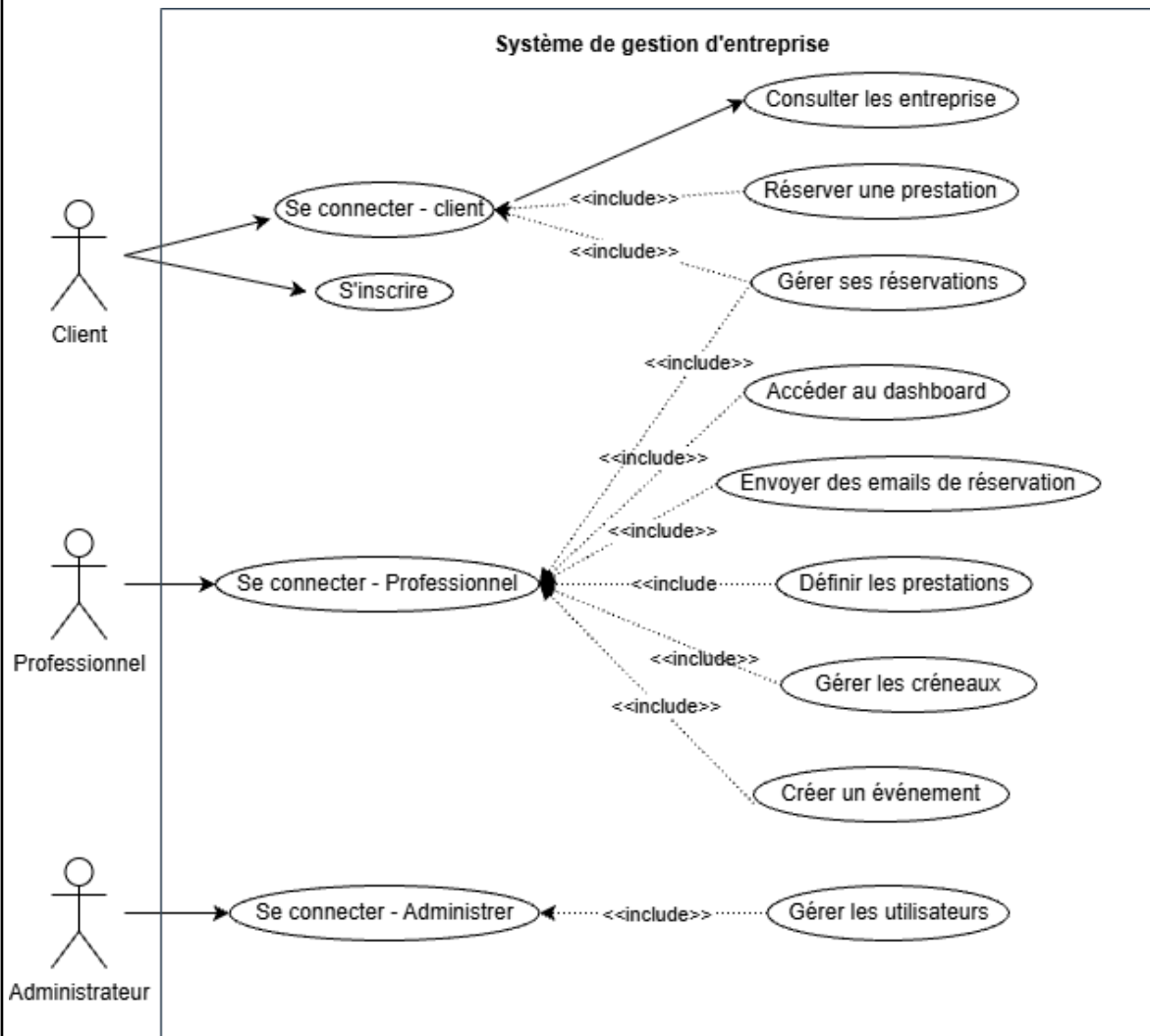
La conception de la base de données a été réalisée selon une démarche progressive. Un modèle Entité-Association (MCD) a d'abord été élaboré afin d'identifier les entités principales du système (utilisateurs, types d'utilisateurs, entreprises, prestations, semaines types, créneaux, réservations, événements et e-mails) ainsi que leurs relations. Ce modèle conceptuel a ensuite été transformé en schéma relationnel implémenté dans PostgreSQL à l'aide de l'ORM SQLAlchemy.

Les principales productions réalisées dans le cadre du projet sont :

- Une API REST fonctionnelle développée avec Flask.
- Une interface web responsive développée avec React.
- Une base de données PostgreSQL avec scripts de création et migrations.
- Une application conteneurisée via Docker et Docker Compose.
- Un dépôt GitHub public contenant le code source.
- Une documentation technique et utilisateur accessible en ligne.

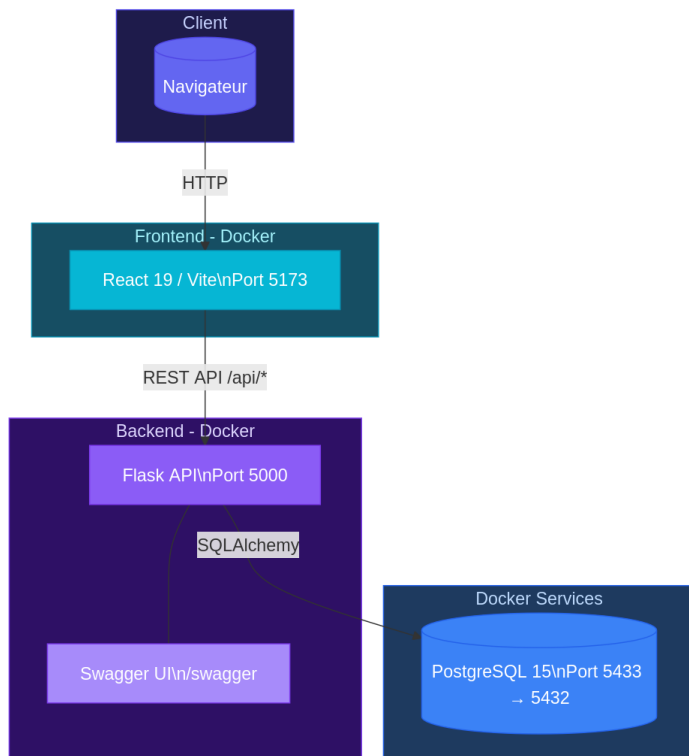
Ce projet met en évidence la maîtrise de l'ensemble du cycle de développement d'une application web, depuis l'analyse du besoin jusqu'à la mise à disposition d'une solution fonctionnelle, en intégrant des problématiques de conception, de sécurité, de performance et d'ergonomie.

Analyse et Conception : Use Case



Développement et Services Web:

L'application exploite les technologies Web pour les échanges de données entre le client (React) et le serveur (Flask) via une **API REST**. L'utilisation de **React Router 7** assure une navigation fluide entre les composants applicatifs.

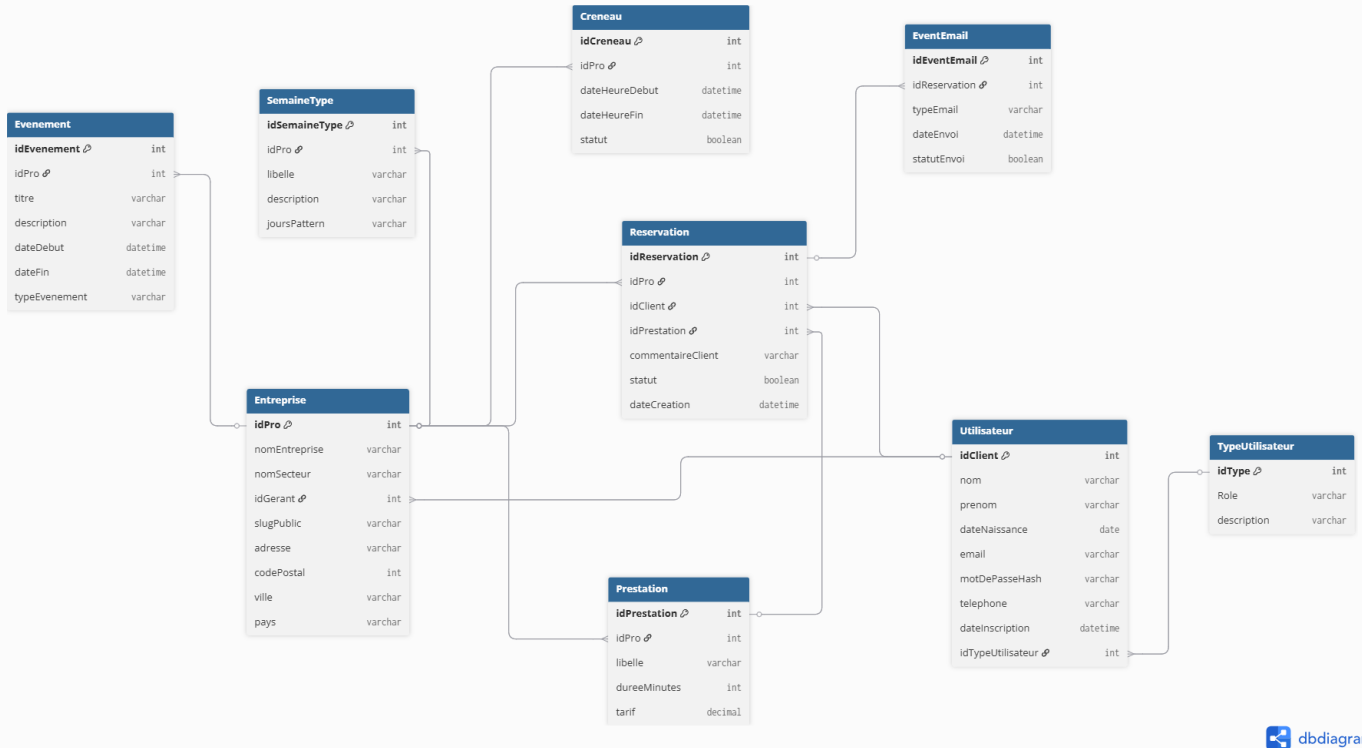


Ce schéma représente une architecture n-tiers (en couches) entièrement conteneurisée avec Docker, ce qui garantit la portabilité et l'isolation des services.

Gestion des données persistantes: Schéma de base de données

Le stockage des données repose sur le système de gestion de base de données PostgreSQL 15. Les interactions avec la base sont effectuées via l'ORM SQLAlchemy, qui permet une abstraction de la couche SQL tout en assurant la sécurité, la structuration et la fiabilité des requêtes.

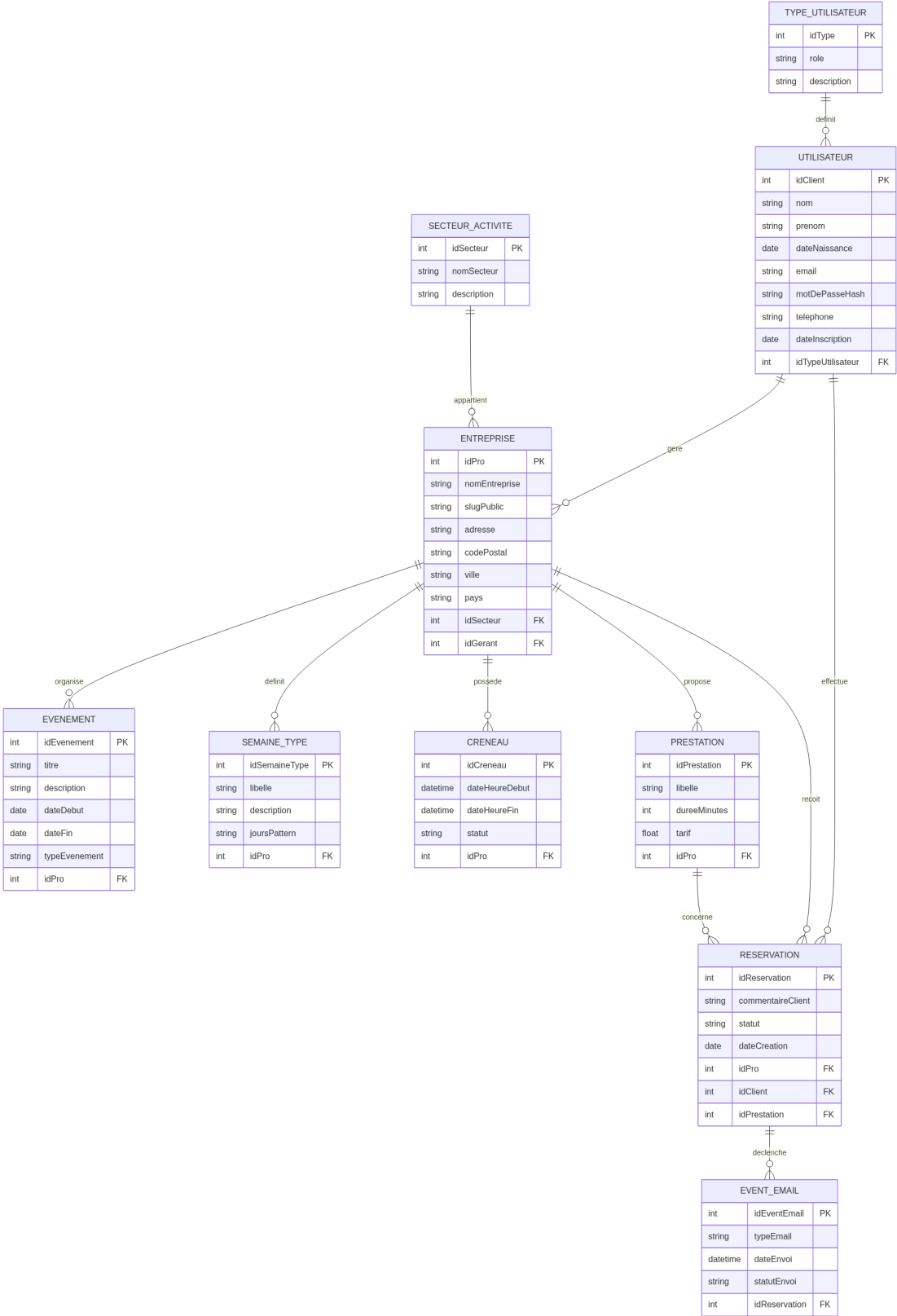
Schéma de base de données :



Ce schéma structure une plateforme de mise en relation B2C. Son architecture repose sur trois piliers :

- **L'intégrité métier** : La table **Reservation** centralise les flux en liant un client, un professionnel et une prestation spécifique.
- **La flexibilité temporelle** : Le couplage entre **Creneau** et **SemaineType** permet une gestion dynamique et automatisée des agendas.
- **La sécurité Native** : La séparation des rôles (**TypeUtilisateur**) et le hachage des secrets garantissent un contrôle d'accès robuste.

MCD — Modèle Conceptuel de Données



MLD — Modèle Logique de Donnée

La base de données est structurée autour de l'entité ENTREPRISE, qui centralise l'offre de services. Chaque entreprise est rattachée à un SECTEUR_ACTIVITE et gérée par un UTILISATEUR (le gérant).

Le fonctionnement repose sur la mise en relation entre l'offre et la demande :

- Le gérant définit ses PRESTATIONS (tarifs, durées) et ses disponibilités via les CRENEAUX et la SEMAINE_TYPE.
- Le client effectue une RESERVATION, qui sert de table pivot en liant un utilisateur, une prestation et une entreprise précise.
- Enfin, une table EVENT_EMAIL est associée à chaque réservation pour assurer le suivi automatique des notifications (confirmations ou rappels).

Cette modélisation garantit l'intégrité des données : on ne peut pas avoir de réservation sans client, ni de prestation sans entreprise parente.

SECTEUR_ACTIVITE (idSecteur, nomSecteur, description)

TYPE_UTILISATEUR (idType, rôle, description)

UTILISATEUR (idClient, nom, prenom, dateNaissance, email, motDePasseHash, telephone, dateInscription, #idTypeUtilisateur)

ENTREPRISE (idPro, nomEntreprise, slugPublic, adresse, codePostal, ville, pays, #idSecteur, #idGerant)

EVENEMENT (idEvenement, titre, description, dateDebut, dateFin, typeEvenement, #idPro)

SEMAINE_TYPE (idSemaineType, libelle, description, joursPattern, #idPro)

CRENEAU (idCreneau, dateHeureDebut, dateHeureFin, statut, #idPro)

PRESTATION (idPrestation, libelle, dureeMinutes, tarif, #idPro)

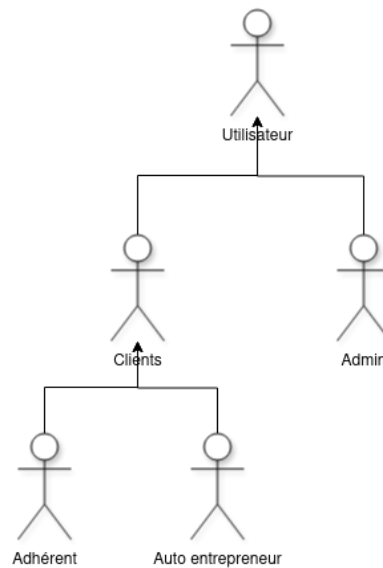
RESERVATION (idReservation, commentaireClient, statut, dateCreation, #idPro, #idClient, #idPrestation)

EVENT_EMAIL (idEventEmail, typeEmail, dateEnvoi, statutEnvoi, #idReservation)

Maintenance et Gestion des Versions

Le projet utilise **Git** comme outil collaboratif pour l'intégration continue. L'historique des **commit** témoigne du suivi rigoureux des itérations de développement, de la correction des dysfonctionnements et de l'évolution des fonctionnalités.

Diagramme droit d'utilisation:



Déploiement et Qualité (Infrastructure):

Afin d'assurer une stabilité optimale et une meilleure maîtrise de l'environnement de production, la solution repose sur une architecture conteneurisée basée sur Docker.

Un Dockerfile, positionné à la racine du projet, orchestre le déploiement des différents composants de l'application — backend, frontend et base de données — au sein de conteneurs distincts, garantissant ainsi isolation, cohérence et portabilité.

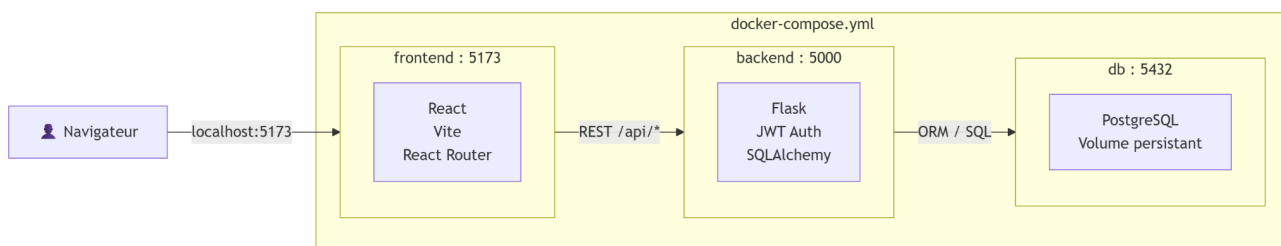
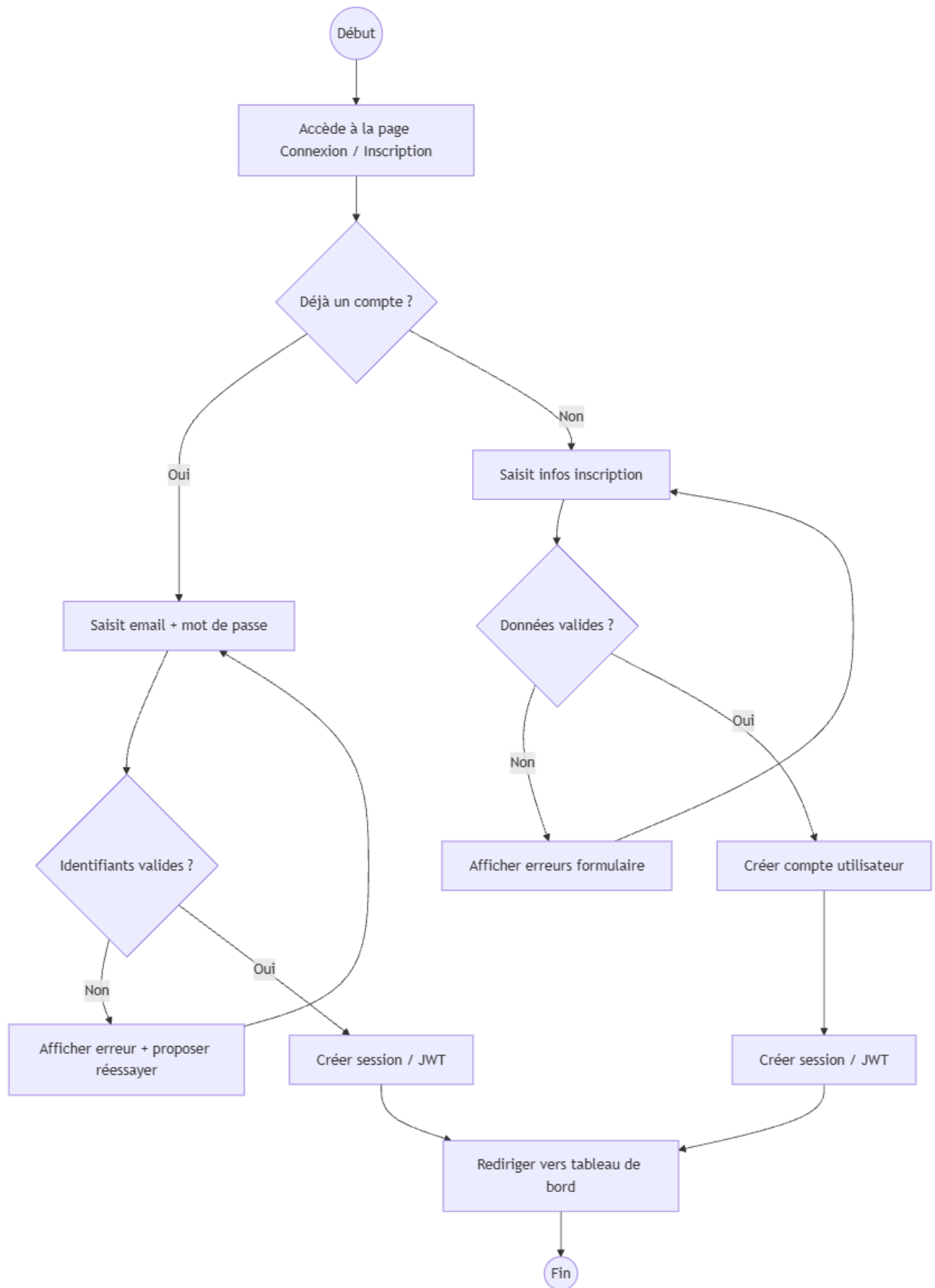
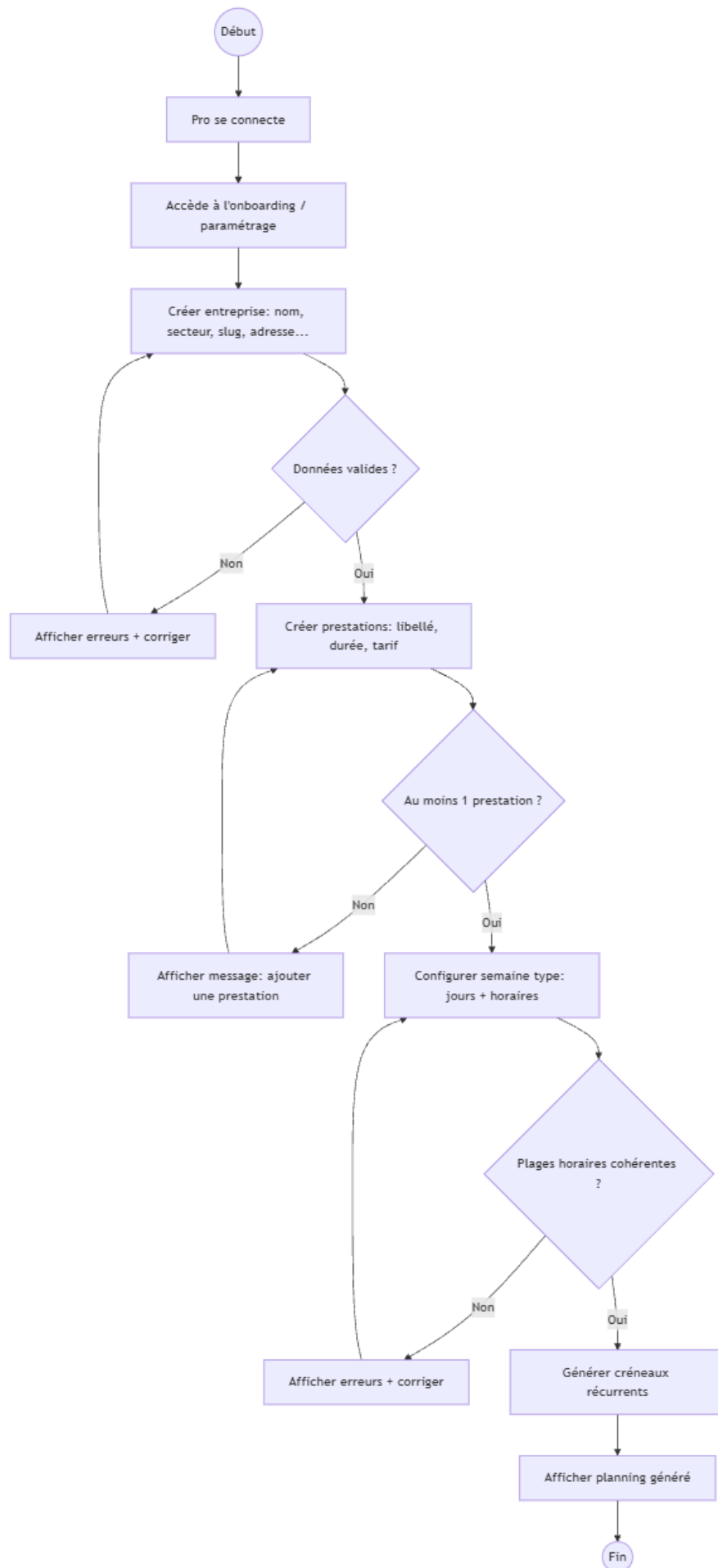


Diagramme d'activités

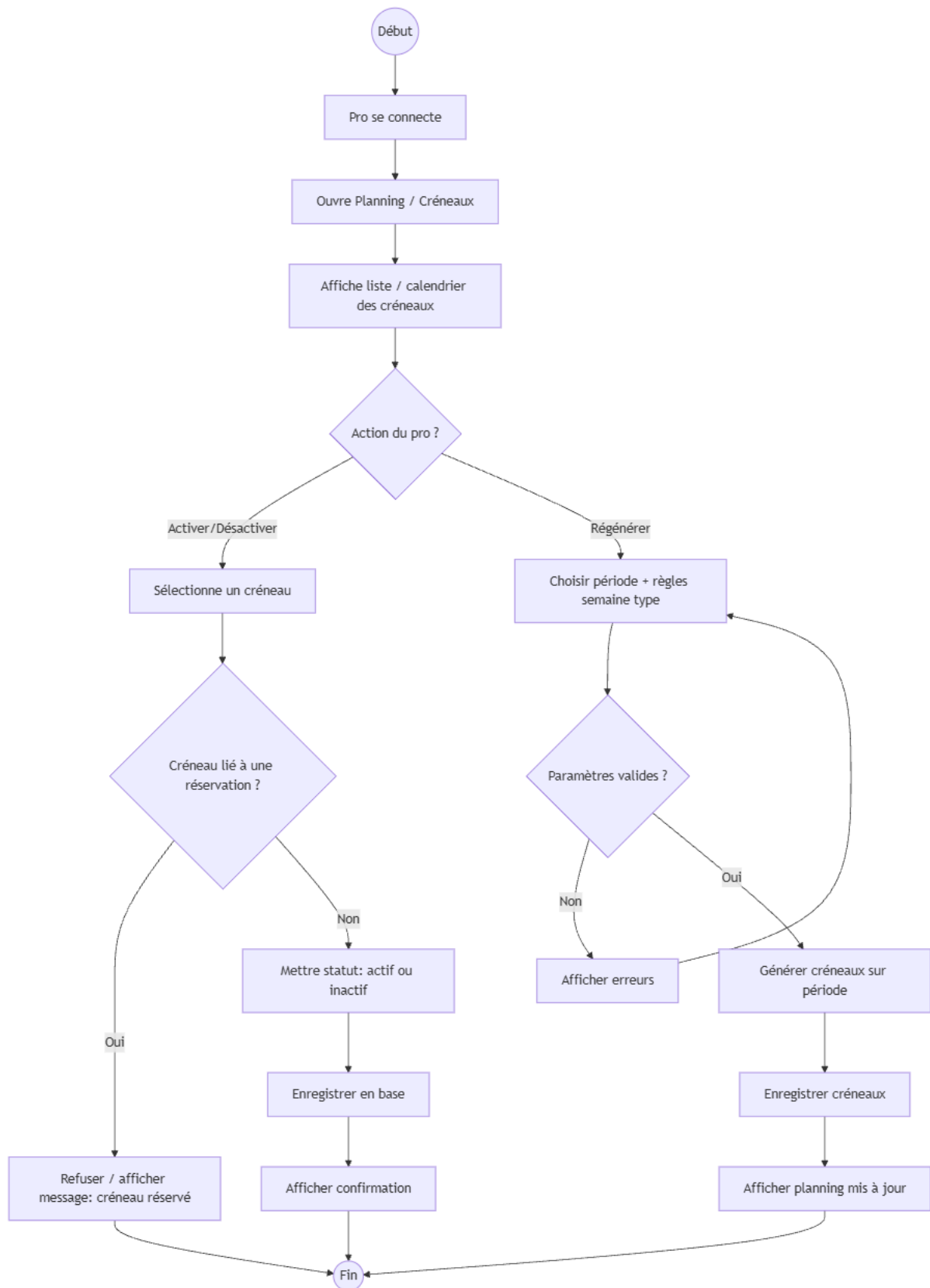
- Inscription/Connexion



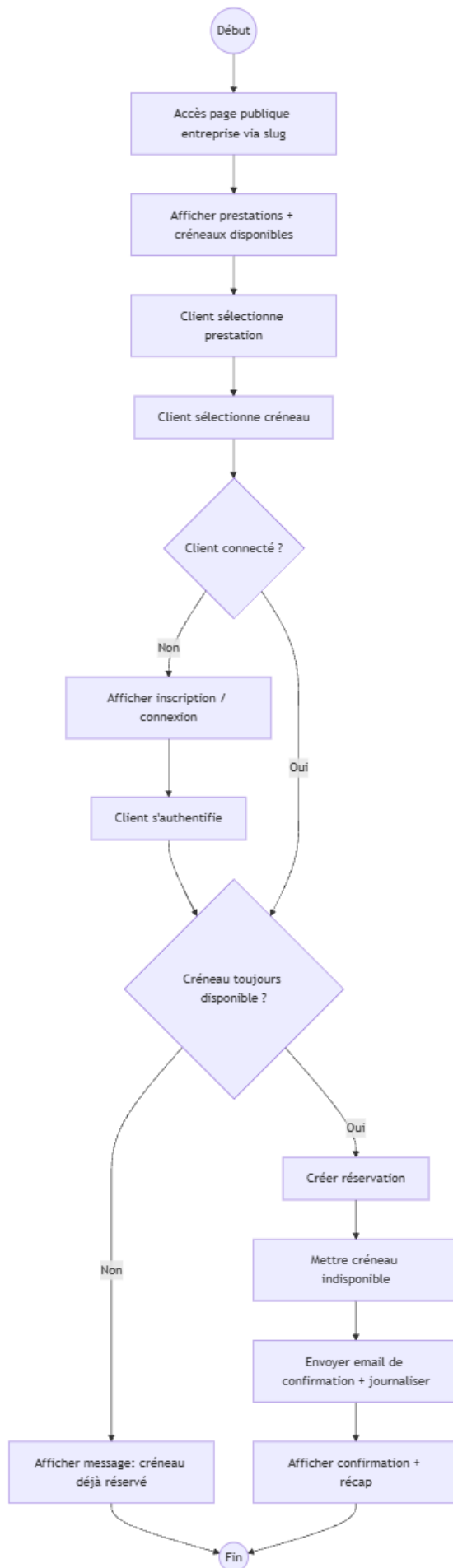
- Configuration initiale Pro (entreprise + prestations + semaine type)



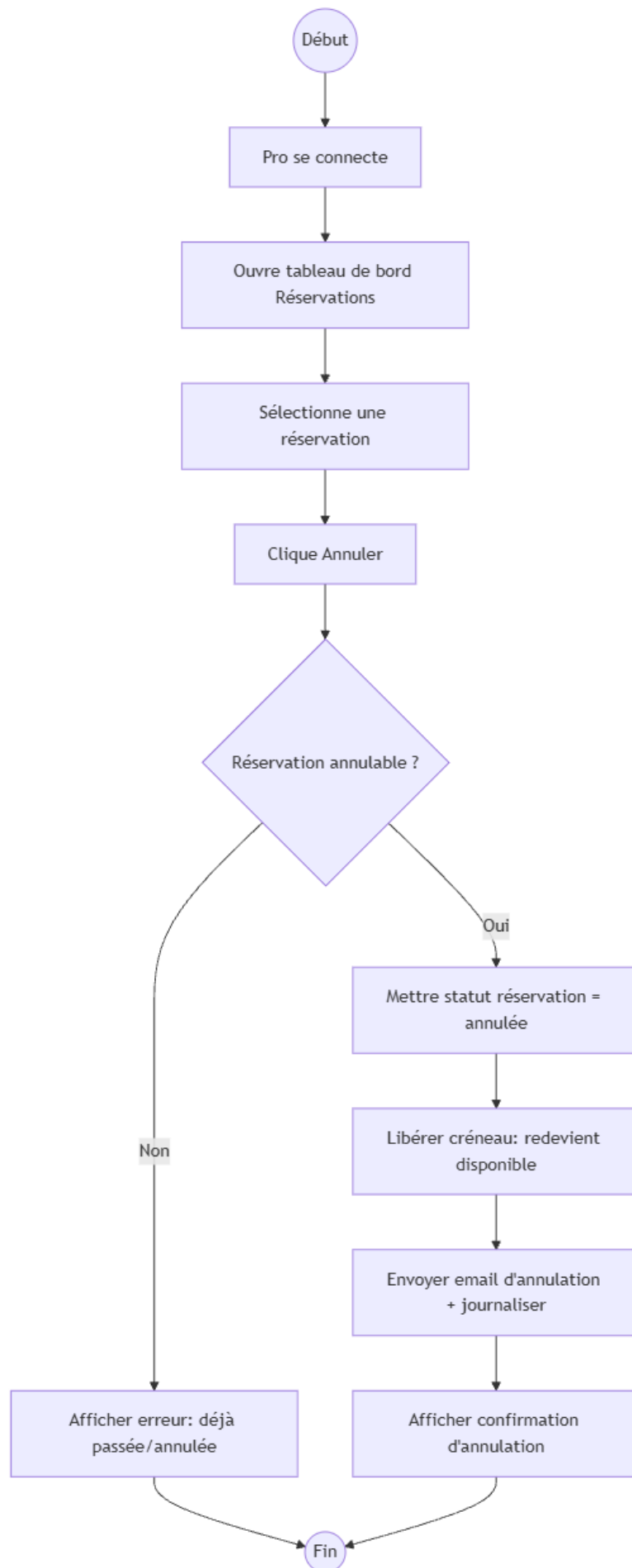
- Génération / gestion des créneaux (activer / désactiver)



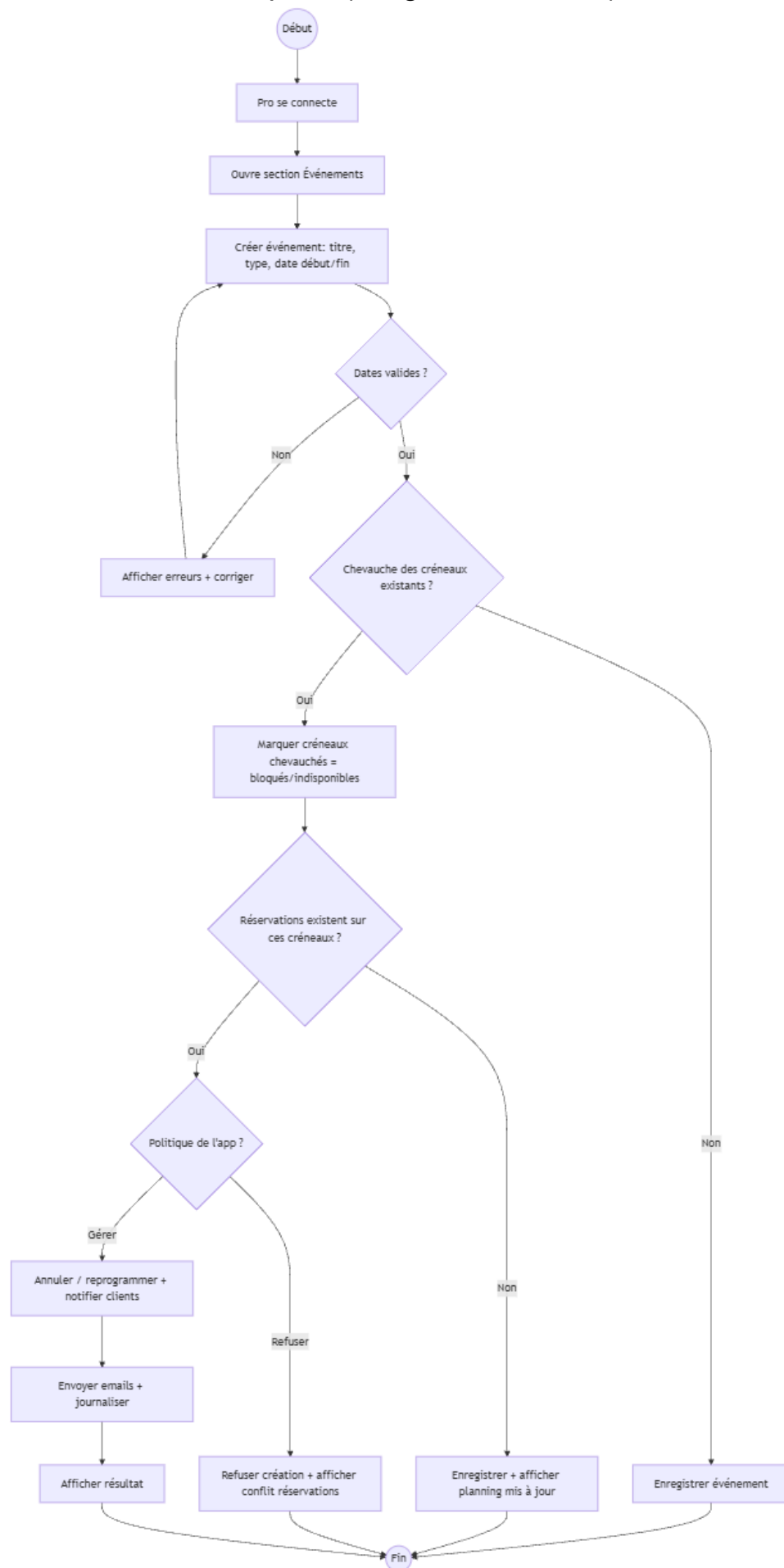
- Réservation d'un rendez-vous (Client)



- Annulation d'une réservation (par le Pro)



- Ajout d'un événement bloquant (congrés / fermeture)



- Ajout manuel d'une réservation (Pro)

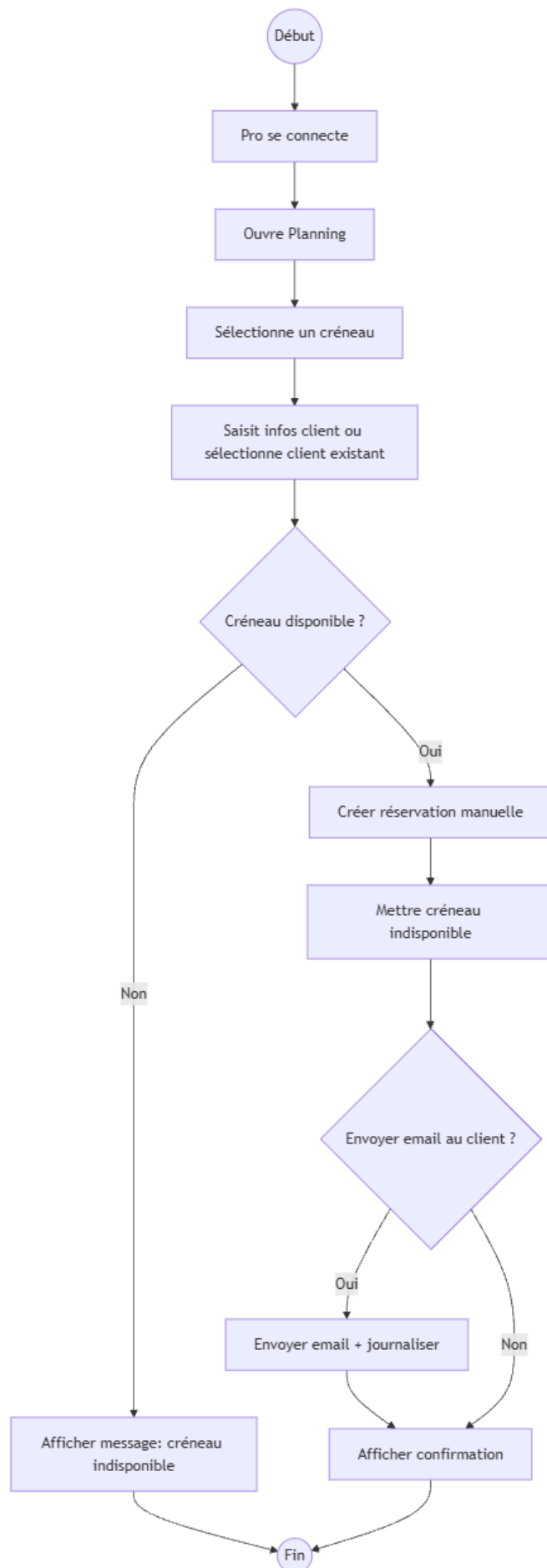
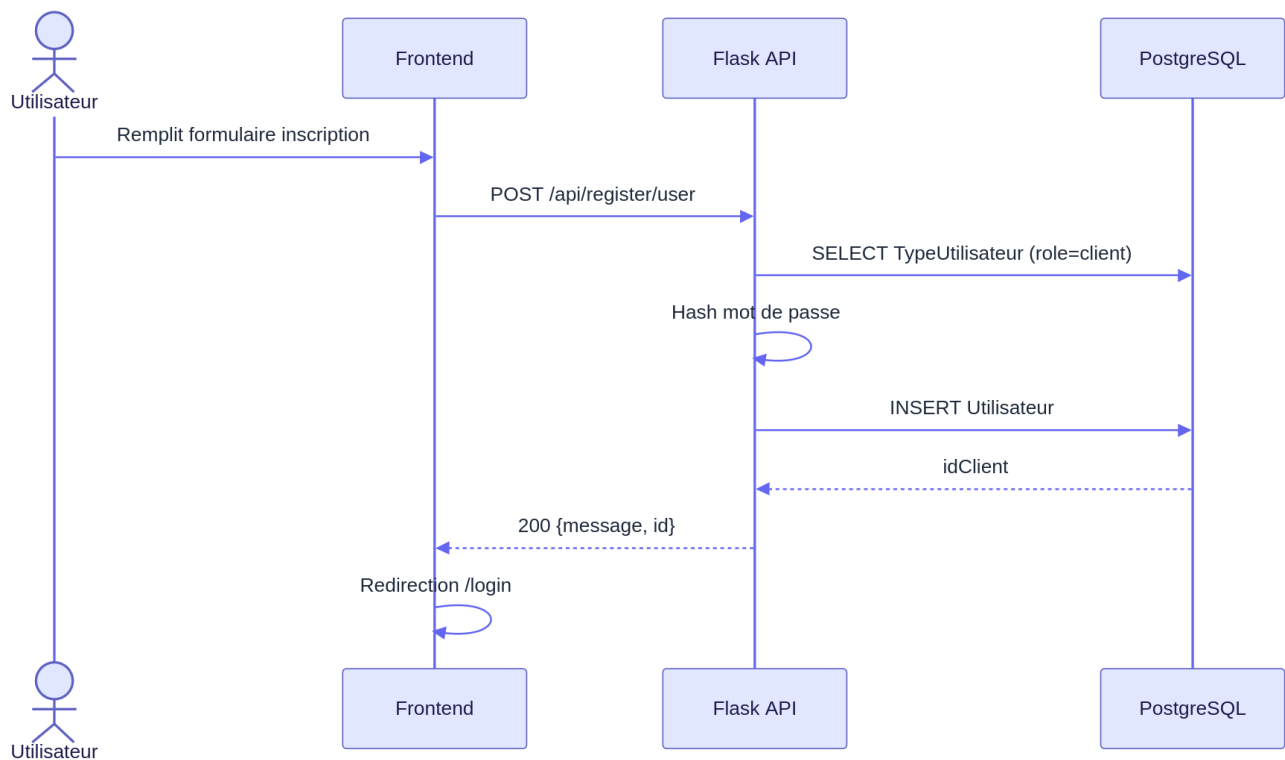


Diagramme de séquence: Authentification

Inscription client:



Connexion / Déconnexion:

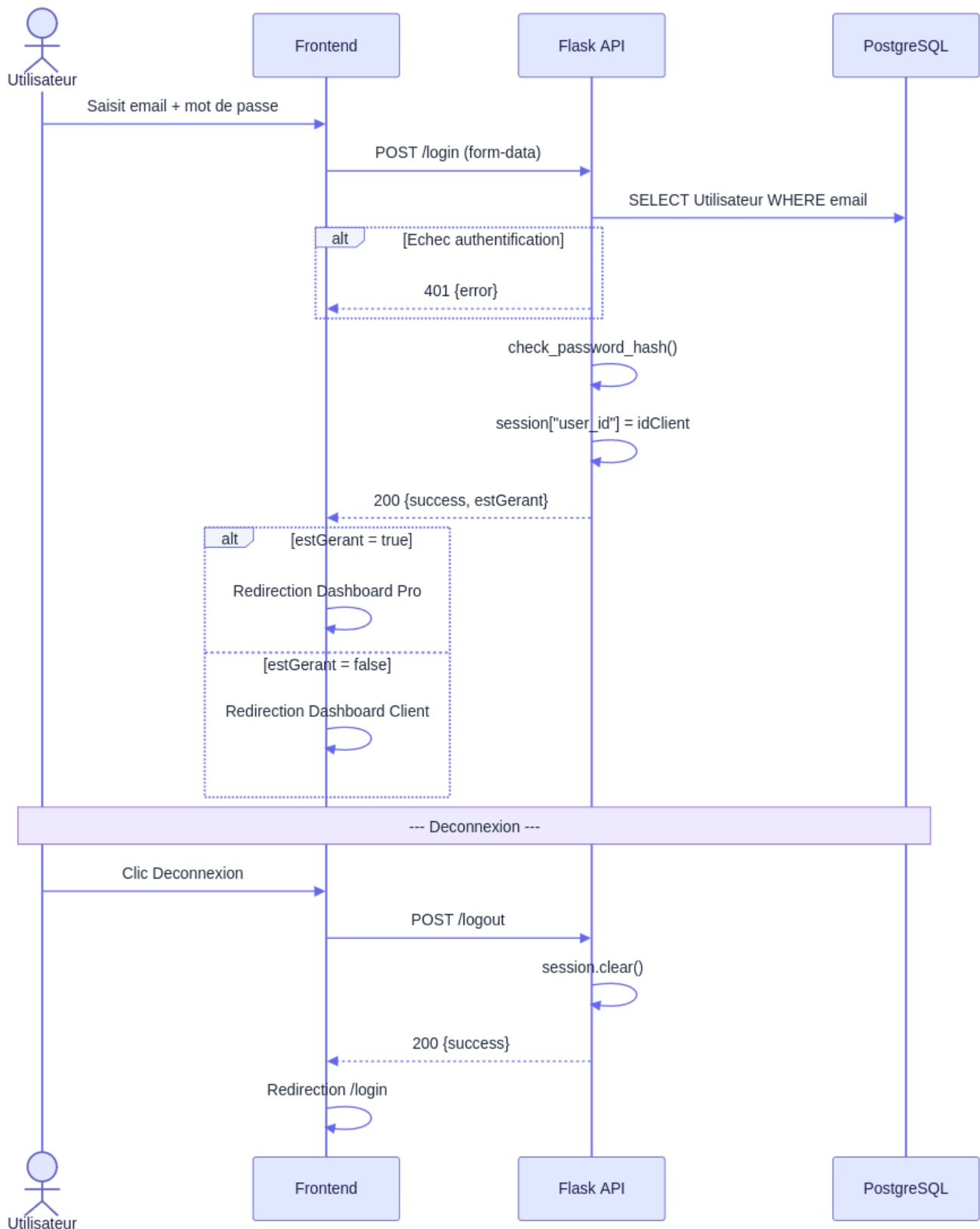


Diagramme de séquence: Recherche

Recherche d'entreprises:

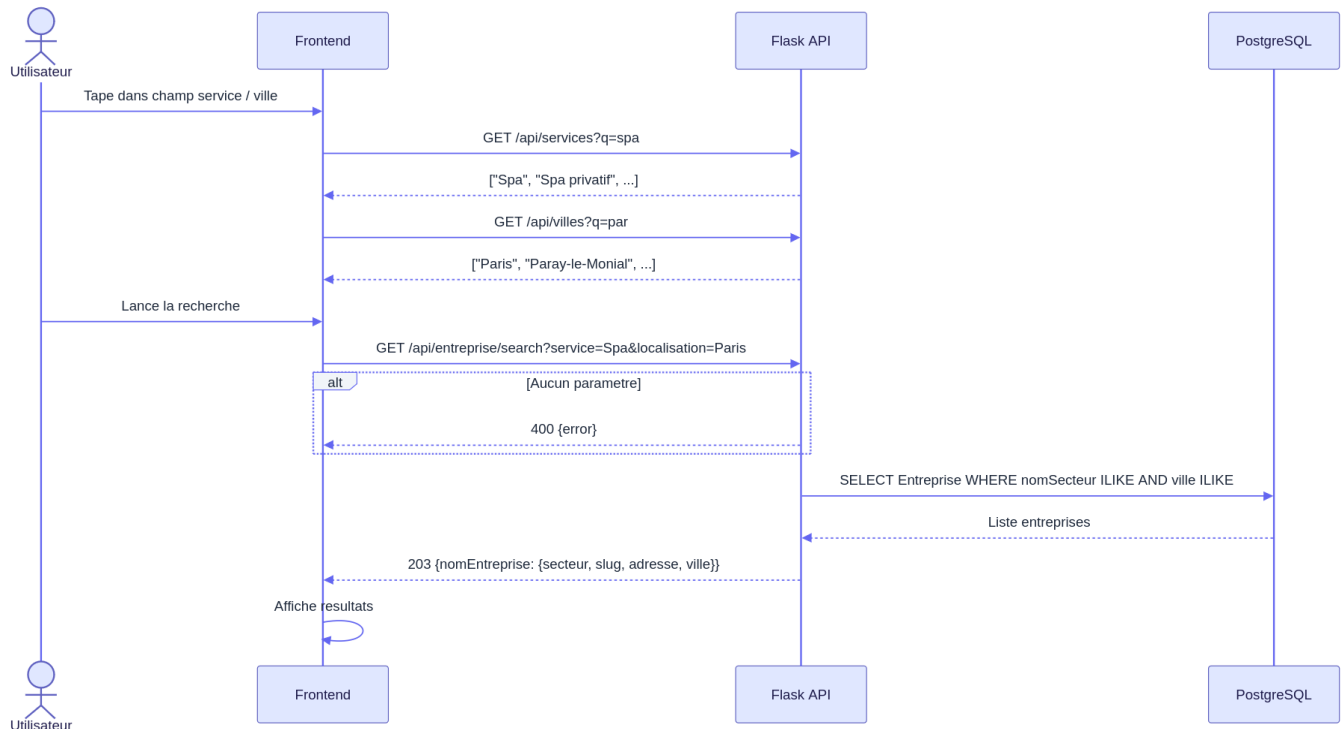


Diagramme de séquence: Dashboard professionnelle

Réservation et calendrier:

